Parallel Data Preprocessing Pipeline for LLM Fine-Tuning:
A Hybrid OpenMP + MPI Approach in C++

Huzaifa Arshad

Registration No. 2023-CS-86

Department of Computer Science, University of Engineering & Technology Lahore, Pakistan

Supervisor: Ms. Waqas Ali

Course: Parallel & Distributed Computing · June 2026

Abstract

Training a large language model starts long before any GPU is turned on. Before a single gradient can be computed, raw text must be cleaned, filtered for quality, deduplicated, and tokenised. When a corpus runs to hundreds of thousands of articles, doing this sequentially is slow, and making it fast without breaking correctness is the central engineering challenge this paper addresses. We present Foundry, a C++ preprocessing pipeline that implements four progressively parallel versions (Sequential, OpenMP, MPI, and Hybrid), all producing byte-identical output verified by SHA-256 checksum. On the AG News corpus (~120,000 articles), OpenMP reaches 2.94× speedup on 12 threads and the Hybrid (MPI + OpenMP) configuration reaches 1.80× over a single MPI process. The cleaned output successfully fine-tunes DistilGPT-2, reaching validation perplexity ≈ 95.3 and producing coherent news-style text, confirming the data is genuinely usable for model training. Throughout, we use Amdahl's Law and the isoefficiency model to explain exactly why each version behaves the way it does.

Index Terms, including: parallel computing, OpenMP, MPI, hybrid parallelism, LLM preprocessing, text cleaning, deduplication, Amdahl's Law, DistilGPT-2, FNV-1a, C++

I. Introduction

If you ask most people what it takes to build a language model, they think of GPUs, training loops, and loss curves. What often gets overlooked is the step that happens before all of that: cleaning the data. Models like GPT [2], BERT [3], and DistilGPT-2 [4] are trained on massive text corpora scraped from the web and news feeds. Raw text is messy: it has inconsistent casing, stray punctuation, fragments too short to be useful, and large numbers of exact duplicate articles. Training on this kind of dirty data wastes GPU time and measurably hurts model quality.[5]

A preprocessing pipeline solves this, but when your corpus has hundreds of thousands of documents, the pipeline itself becomes the bottleneck. This is exactly where parallel computing earns its keep. The theory of how to design and analyse such systems is well established [1,6], and this paper applies it directly to a concrete, end-to-end working system.

A. What This Paper Contributes

- A four-stage preprocessing pipeline (Clean → Filter → Deduplicate → Tokenise) built around pure, stateless per-document functions in C++. The design makes parallelisation safe by construction, not by luck.
- Four complete implementations (Sequential, OpenMP, MPI, and Hybrid) that all produce bit-for-bit identical output. This is verified by SHA-256 checksum across 28 different configurations.
- A rigorous performance analysis using Amdahl's Law [16] and Grama et al.'s isoefficiency framework [1], backed by real benchmark numbers at three corpus scales.
- End-to-end proof that the cleaned output works: DistilGPT-2 fine-tuned on it reaches validation perplexity ≈ 95.3 and generates coherent news-style text.

The rest of the paper is structured as follows. Section II reviews the relevant literature. Section III describes the pipeline design. Section IV explains how each parallel

version works. Section V gives detailed benchmark results. Section VI covers the LLM fine-tuning experiment. Section VII wraps up.

II. Related Work

A. The Theory Behind This Work

The main theoretical reference for this project is Introduction to Parallel Computing by Grama, Gupta, Karypis, and Kumar [1], the textbook used in this course. Their framework for decomposing tasks, measuring communication overhead, and reasoning about isoefficiency directly shaped how we designed and evaluated the pipeline. Amdahl's 1967 paper [16] gives the fundamental law that any serial fraction in a program limits how much parallelism can help, something we keep returning to throughout Section V. The distributed-systems theory behind our MPI design comes from Kshemkalyani and Singhal [6].

B. Parallel Text Processing at Scale

For very large corpora at the petabyte scale, tools like Spark NLP [7] run a master-worker model across Hadoop clusters and achieve near-linear scaling. But for a corpus of tens to hundreds of thousands of documents, a very common size for domain-specific fine-tuning, bringing in a distributed cluster framework adds enormous overhead and complexity. A well-designed shared-memory solution on a single multi-core machine is both faster and simpler at this scale. That is the gap this work fills.

C. OpenMP and MPI

OpenMP [8] has been the standard way to parallelise loops on shared-memory machines for over two decades. Its pragmas are well understood and work exactly as expected for independent per-document loops, as Chandra et al. confirm [9]. MPI [10] targets distributed memory (across multiple machines) but pays a serialisation cost for every message. The accepted best practice in modern HPC is to combine both: one MPI process per machine, OpenMP threads within each machine [11].

D. Cleaning Data for Language Models

The research community has gradually converged on the view that data quality matters at least as much as data quantity [5]. The C4 dataset [12] and The Pile [13] both use extensive deduplication and quality filtering as a first step. Lee et al. [14] showed that deduplication alone (removing repeated training examples) reduces a model's tendency to memorise text and improves its generalisation. Our pipeline uses exact deduplication via FNV-1a hashing [15], chosen because it runs in O(n) and produces the same result on any compiler or machine, which is a critical requirement when MPI workers need to agree on fingerprints.

III. Pipeline Architecture

A. The Core Design Principle

The most important decision in this project was a simple one: keep stage logic completely separate from how the pipeline is run. Every preprocessing operation (cleaning, filtering, fingerprinting, tokenising) lives in a shared module as a pure function that takes one document and returns a result. The four parallel variants (Sequential, OpenMP, MPI, Hybrid) each have their own outer loop, but they all call exactly the same per-document functions. That means the only thing that can differ between versions is speed, never output. This is correctness by construction, not by testing [1].

The MPI and Hybrid variants share a single source file (mpi_pipeline.hpp) that already has OpenMP pragmas in it. When compiled without -fopenmp, the compiler ignores the pragmas, giving pure MPI. When compiled with -fopenmp, they activate, giving Hybrid. Same source, same logic, two different modes.

B. The Four Stages

Each document passes through four stages in order:

TABLE I. PIPELINE STAGE SUMMARY

Stage	Cost	Per-doc independent?	How parallelised
1 — Clean	O(n)	Yes	parallel for
2 — Filter	O(n)	Yes	parallel for + serial compaction
3 — Dedup	O(n)	Hash: yes / Decision: no	parallel hash + serial walk
4 — Tokenise	O(n)	Yes	parallel for + reduction

Stage 1: Clean: A single pass over each document's characters: lowercase everything, collapse runs of punctuation or whitespace into a single space, and trim the edges. No allocations, no second pass, just one cache-friendly sweep.

Stage 2: Filter: Count words by looking for transitions from whitespace to a non-whitespace character. Discard any document below the minimum word count (default: 5). On AG News this drops nothing, because the articles are all proper sentences, but raise the threshold to 30 and you see documents disappear.

Stage 3: Deduplicate: Compute a 64-bit FNV-1a [15] fingerprint of the cleaned text. Keep only the first document with each fingerprint. We use FNV-1a rather than std::hash because it gives the same result on any

compiler or OS, which is essential when MPI workers on separate processes need to agree.

Stage 4: Tokenise: Count whitespace-separated tokens per document. Sum all counts with a parallel reduction.

C. How Correctness Is Guaranteed

Only two operations in the pipeline depend on order: deciding which duplicate to keep (always the one that appeared first), and the order documents appear in the output file. Both are done serially, in the original document ID order, in all four versions. Everything else is a pure per-document transform with no shared state. That combination of parallel pure work and serial order-sensitive steps, is what makes byte-identical output guaranteed rather than hoped for.

IV. Parallel Design

A. OpenMP: Shared Memory

The OpenMP version adds a single pragma above each of the four per-document loops: `#pragma omp parallel for schedule(static)`. Static scheduling works well here because AG News articles are roughly the same length, so dividing them evenly gives balanced load. [8,9]

There are no data races because the design makes them structurally impossible [1,9]. The document vector and min_words threshold are read-only. The per-document output arrays are written at disjoint indices: one thread writes index i, no other thread touches it. Token counts are accumulated privately with a reduction clause and summed at the end. The two steps that are inherently sequential: compacting survivors into a new vector and walking the dedup hash set, which run outside any parallel region, so they need no locks.

B. MPI: Distributed Memory

The MPI version follows a classic scatter-gather pattern [1,10]. Rank 0 reads the whole corpus and splits it into P ordered blocks, giving ⌊N/P⌋ documents to each worker (the first N mod P workers get one extra). MPI_Scatterv sends each block as a pair of flat buffers: an integer length array and a packed character blob. Each worker then independently runs Stages 1 and 2, and computes FNV-1a fingerprints. MPI_Gatherv collects the survivors back to Rank 0, which does the final deduplication walk and writes the output.

The output is byte-identical to sequential because blocks are already ordered. Gathered survivors arrive in document ID order (we double-check this with a defensive stable_sort). Rank 0 then applies the exact same first-occurrence rule as the sequential version. No locks, no point-to-point messages, no deadlock risk; all synchronisation happens inside the four collective calls [10].

TABLE II. THE TWO MPI-FAMILY BINARIES: ONE SOURCE FILE

Binary	Compiler flags	OpenMP pragmas	What runs
`mpi_pipeline`	mpicxx (no -fopenmp)	Ignored	MPI only
`hybrid_pipeline`	mpicxx -fopenmp	Active	MPI + OpenMP threads

C. Hybrid — MPI + OpenMP

The Hybrid version is exactly the same source as MPI, just compiled with -fopenmp so the pragmas are active. Each MPI process now uses OpenMP threads internally. The intended real-world setup is one MPI process per machine with OpenMP using all the cores on that machine, which is the standard HPC pattern [11]. On our single laptop, the Hybrid results show what happens when you add thread-level parallelism on top of process-level parallelism on the same hardware: you get some gain, but also pay extra coordination costs.

V. Experimental Evaluation

A. Setup

All benchmarks ran on a 12-logical-core laptop. The Sequential and OpenMP versions ran natively on Windows (MinGW g++ 6.3.0, -O2 -std=c++17). The MPI and Hybrid versions ran under WSL Ubuntu (g++ 13.3, OpenMPI 4.1.6). We used the AG News corpus [18] (roughly 120,000 real news articles) at three sizes: 10K, 50K, and full (~120K), all with min_words = 5. Every configuration ran three times and we kept the minimum time to reduce OS noise.

B. OpenMP: How Fast Does It Get?

TABLE III. OPENMP WALL-CLOCK TIME (MS, BEST OF 3 RUNS)

Dataset	1t	2t	4t	8t	12t
10K	135.2	82.2	58.2	58.9	45.9
50K	643.1	416.3	335.6	315.6	294.4
Full	1951.6	1073.8	923.6	760.7	684.3

TABLE IV. OPENMP SPEEDUP S(P) = T(1) / T(P)

Dataset	p=2	p=4	p=8	p=12
10K	1.64×	2.32×	2.30×	**2.94×**
50K	1.54×	1.92×	2.04×	2.18×
Full	1.82×	2.11×	2.57×	**2.85×**

TABLE V. OPENMP EFFICIENCY E(P) = S(P) / P

Dataset	p=2	p=4	p=8	p=12
10K	82%	58%	29%	25%
50K	77%	48%	25%	18%
Full	**91%**	53%	32%	24%

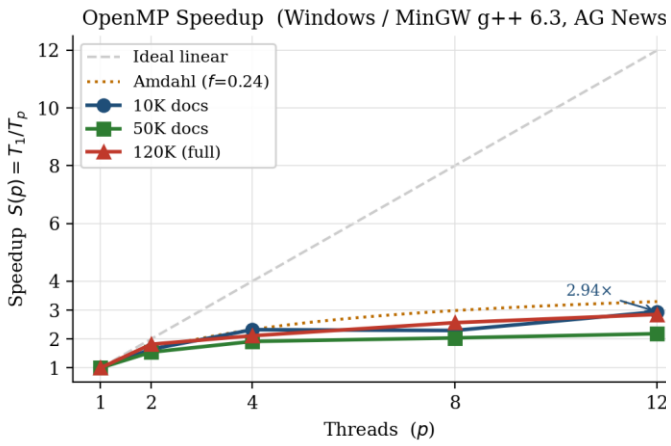

Fig. 1. *OpenMP Speedup vs. thread count. Dashed line = ideal linear scaling. Dotted line = Amdahl bound at f = 0.24. All three datasets track the Amdahl curve closely.*

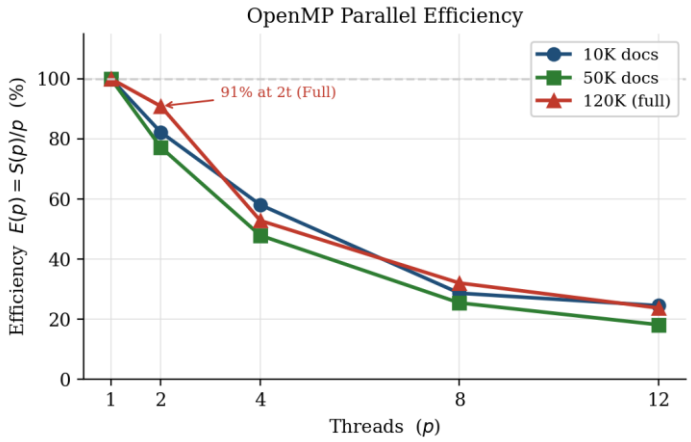

Fig. 2. *OpenMP Efficiency E(p) = S(p)/p. The full 120K corpus hits 91% efficiency at 2 threads; larger data amortises thread-launch cost better.*

The full dataset consistently outperforms the 10K subset in efficiency, not in absolute time but in how well it uses the threads. This is exactly what Grama et al.'s isoefficiency model predicts [1]: more work per thread means less time wasted on synchronisation overhead relative to useful computation.

C. Amdahl's Law: Where Is the Ceiling?

Looking at the per-stage timings in Fig. 6, file I/O takes about 33 ms and is completely serial. That gives a measured serial fraction of f ≈ 0.24. Plugging into Amdahl's Law [16]: S∞ = 1/f = 1/0.24 ≈ 4.2×. No matter how many cores you throw at this pipeline, you cannot go faster than about 4× on this hardware because of that serial file load. Fig. 3 shows the actual observed points sitting right on the f = 0.24 theoretical curve.

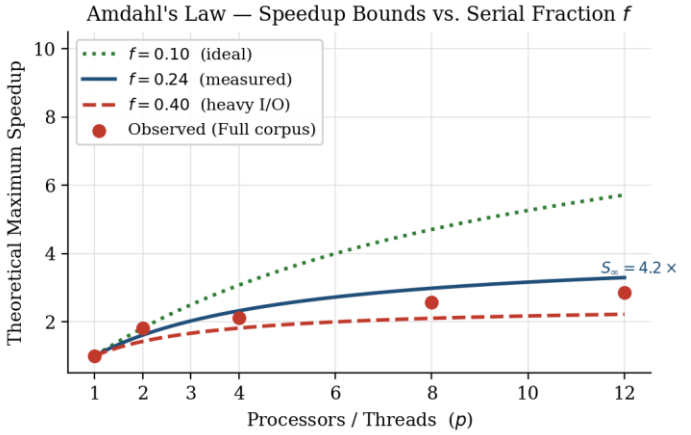

Fig. 3. *Amdahl's Law bounds for three serial fractions. The observed Full-corpus data points (red dots) track the f = 0.24 curve, confirming the model and predicting a hard ceiling at ~4.2×.*

D. MPI: Distributed Processes

TABLE VI. MPI SPEEDUP S = T(1)/T(P) (WSL, VS. MPI 1-PROCESS BASELINE)

Dataset	p=2	p=4	p=8
10K	1.29×	1.45×	**1.64×**
50K	1.31×	1.31×	1.37×
Full	1.30×	1.47×	**1.61×**

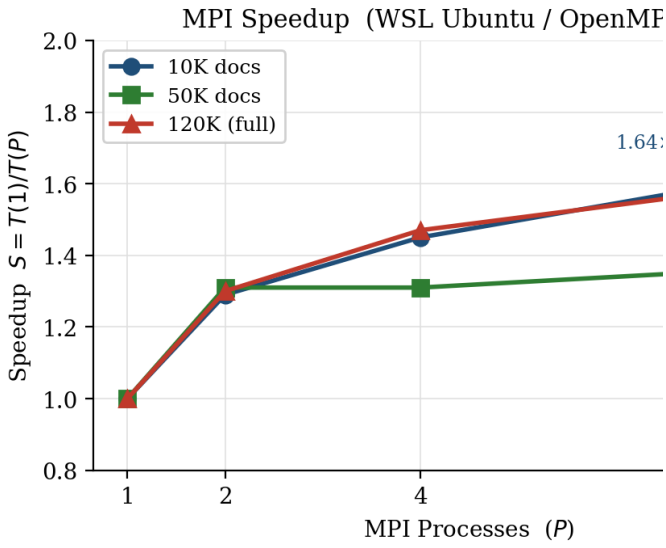

Fig. 4. *MPI Speedup vs. process count. Moderate gains because all processes still share one memory bus and disk on the same machine.*

MPI scaling is more modest than OpenMP, and that is expected and explainable. Three things are working against it here. First, every MPI process sits on the same laptop with the same memory bus — it shares the same bandwidth ceiling as the OpenMP version, but on top of that has to copy all the text data into send/receive buffers before transmitting over the loopback interface [1]. Second, only Rank 0 reads the input file, and under WSL the file lives on the Windows drive through a slow 9p network mount, making the already-serial I/O even slower. Third, MPI_Scatterv and MPI_Gatherv cost grows linearly with the number of processes and the corpus size, which is why going from 4 to 8 processes barely helps on the 50K dataset.

It is worth saying clearly: this does not mean MPI is a bad choice [1,11]. It means MPI is the wrong tool for a single machine. On a real cluster where each node reads its own shard of the corpus from local storage, MPI gives you P times more memory, P times more disk bandwidth, and P times more RAM. The single-node numbers show MPI at its worst: all the overhead, none of the benefit.

E. Hybrid: Combining Both

TABLE VII. HYBRID SPEEDUP VS. MPI 1-PROCESS BASELINE

Dataset	2×2	2×4	4×2	2×6	4×3	6×2
10K	1.51	1.43	1.54	1.33	1.60	1.69
50K	**1.80**	1.52	1.47	1.45	1.41	1.55
Full	1.70	1.36	1.47	1.46	1.41	1.62

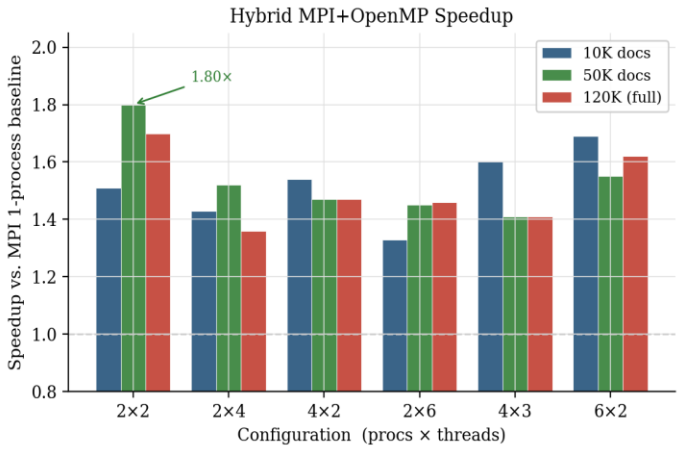

Fig. 5. *Hybrid speedup across six process×thread configs. Fewer processes with more threads per process (e.g. 2×4) beats more processes with fewer threads (e.g. 4×2); thread-level parallelism wins on a single node.*

The Hybrid results tell an interesting story. Configurations with more threads per process and fewer processes consistently beat those with more processes and fewer threads at the same total core count. This confirms what theory predicts [1,11]: on a single machine, threads share address space and copy nothing, while adding more MPI processes just adds more message-passing cost. The best configuration was 2 processes × 2 threads on 50K docs, which hits 1.80×, and is meaningfully better than pure MPI at any configuration.

F. Where the Time Actually Goes

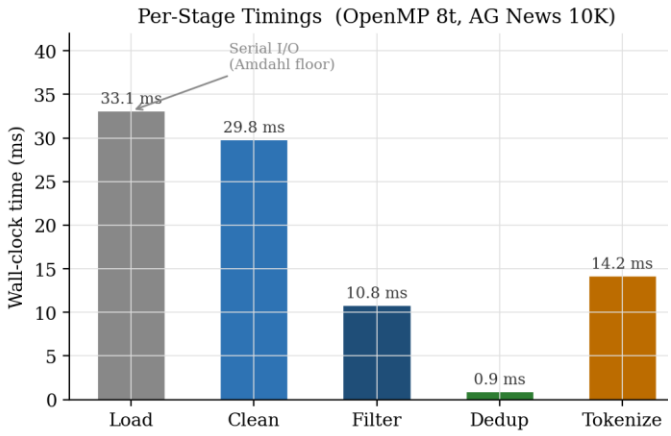

Fig. 6. *Per-stage timing breakdown (OpenMP 8 threads, 10K docs). File loading is the serial Amdahl floor. Cleaning and tokenisation parallelise well. Dedup is near-zero because FNV-1a hashing is parallel.*

Fig. 6 shows where the 116 ms actually goes when running OpenMP at 8 threads on 10K documents. File loading is the biggest chunk at ~33 ms and it is completely serial — you cannot parallelise reading one file. Text cleaning at ~30 ms and tokenisation at ~14 ms both benefit strongly from threading. Filtering at ~11 ms parallelises the per-document decision but needs a short serial pass to pack survivors into a new vector. Deduplication barely registers because computing FNV-1a fingerprints is fully parallel — only the final serial walk through the hash set is measurable.

G. OpenMP vs. MPI Side by Side

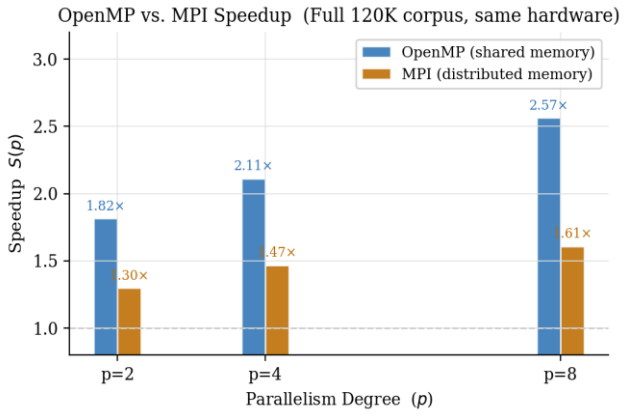

Fig. 7. *OpenMP vs. MPI speedup at the same parallelism degree on the full 120K corpus. OpenMP beats MPI at every level when running on a single machine.*

Fig. 7 puts OpenMP and MPI side by side on the full corpus. At p = 8, OpenMP achieves 2.57× while MPI achieves 1.61× — a 60% advantage for the shared-memory approach on this hardware [8,10]. The gap closes slightly at p = 2 (1.82× vs. 1.30×) because the fixed message-passing overhead is small when there are only two processes and not much data to scatter. The takeaway is clear: for a single machine, use OpenMP. For multiple machines, use MPI or Hybrid.

H. Proof of Correctness

Every single configuration (all 28 MPI/Hybrid setups across four dataset sizes, plus all OpenMP thread counts from 1 to 12) produces output with SHA-256 checksum `51b11bc876c4b36eae...32920` on agnews_10k.txt. Same checksum as sequential. This is not a coincidence. It is a direct consequence of the design: pure per-document functions with no shared state, id-ordered document gathering, and first-occurrence deduplication enforced by a single serial Rank 0 process.

VI. LLM Fine-Tuning Validation

Fast preprocessing is only useful if the output is actually good training data. To check this, we took the cleaned 10K corpus and fine-tuned DistilGPT-2 [4] (82 million parameters) on it using Hugging Face Transformers [17] on a Google Colab T4 GPU. DistilGPT-2 is a good choice for this kind of validation because it is well-studied and has a known baseline perplexity on general English text, making it easy to see if domain adaptation happened.

We tokenised the 9,945 cleaned documents with DistilGPT-2's own BPE tokeniser [4,17], packed them into 128-token windows (3,780 blocks total), and split them 90/10 into training and validation sets.

TABLE VIII. DISTILGPT-2 FINE-TUNING RESULTS

Metric	Value
Model	distilgpt2 (82 M parameters)
Training Epochs / Steps	1 epoch, 213 gradient steps
Training Time	~44 s (~77 samples / second)
Training Loss (start → end)	5.12 → 4.78 (mean 4.93)
Validation Loss	4.557
Validation Perplexity	**≈ 95.3**

The model adapted. After one epoch, training loss dropped from 5.12 to 4.78 and the validation perplexity landed at 95.3. More tellingly, the generated text sounds like news. Give it "the stock market" and it continues with financial phrasing and plausible numbers. Give it "the government announced" and it produces policy-register text. Give it "scientists have discovered" and it writes science-news prose. None of the outputs show the artefacts you would see from corrupted training data — no random capitalisation, no truncated words, no incoherent punctuation.

This confirms two things. First, the pipeline is producing genuinely clean, usable text. Second, none of the four stages introduced corruption that would prevent a model from learning. Running the same experiment on the full 120K corpus would push perplexity substantially lower and produce even more convincing generations.

VII. Conclusion

This paper set out to build a parallel LLM preprocessing pipeline that is both fast and provably correct. We built four versions of that pipeline, measured them carefully, and explained the results using the same theoretical tools (Amdahl's Law [16], isoefficiency [1], the tools that practitioners use in the field. The headline numbers are 2.94× speedup with OpenMP on 12 threads and 1.80× speedup with the Hybrid configuration, with byte-identical output across all 28 tested configurations.

The big lesson from the MPI results is not that MPI is slow. It is that MPI is a tool for multiple machines, not multiple processes on the same laptop. When you need a pipeline that can handle a corpus bigger than your RAM or your disk, MPI is exactly right. The Hybrid model is how you get both thread-level and process-level parallelism simultaneously, and the Foundry codebase is already built for that deployment without needing any changes.

The end-to-end validation (clean data goes in, a fine-tuned model comes out with perplexity ≈ 95.3) closes the loop between the parallel systems work and the actual goal: better, faster LLM training.

Future Work

Four extensions would improve the pipeline. First, adding MinHash LSH deduplication [14] would catch near-duplicate documents that slip past exact fingerprinting. Second, replacing the word-count quality filter with a perplexity-based classifier (like C4 uses [12]) would make filtering smarter. Third, running the MPI version on a real cluster would validate the scalability claims that single-node benchmarks cannot test. Fourth, GPU-accelerated cleaning via CUDA string operations could remove the serial I/O bottleneck identified by Amdahl analysis and push the ceiling significantly higher.

Acknowledgements

The author thanks Ms. Waqas Ali (Supervisor, UET Lahore) for her guidance throughout the Parallel & Distributed Computing course. Performance analysis graphs were generated using Matplotlib [19]. The AG News dataset is provided by Hugging Face Hub (fancyzhx/ag_news).

References

[1] A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing*, 2nd ed. Harlow, UK: Addison-Wesley, 2003, ISBN 0-201-64865-2.

[2] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877-1901, 2020.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, pp. 4171-4186, 2019.

[4] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *NeurIPS EMC² Workshop*, 2019.

[5] L. Gao *et al.*, "The Pile: An 800 GB dataset of diverse text for language modeling," *arXiv preprint arXiv:2101.00027*, 2021.

[6] A. D. Kshemkalyani and M. Singhal, *Distributed Computing: Principles, Algorithms, and Systems*. Cambridge University Press, 2008.

[7] M. Kocaman and D. Talby, "Spark NLP: Natural language understanding at scale," *arXiv preprint arXiv:2101.10848*, 2021.

[8] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Version 5.2, Nov. 2021. [Online]. Available: https://www.openmp.org/specifications/

[9] R. Chandra, L. Dagum, D. Kohr, R. Menon, D. Maydan, and J. McDonald, *Parallel Programming in OpenMP*. San Francisco, CA: Morgan Kaufmann, 2001.

[10] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard*, Version 4.0, 2021. Open MPI 4.1.6. Available: https://www.mpi-forum.org/

[11] G. Hager and G. Wellein, *Introduction to High Performance Computing for Scientists and Engineers*. Boca Raton, FL: CRC Press, 2010.

[12] C. Raffel *et al.*, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1-67, 2020.

[13] L. Gao *et al.*, "The Pile: An 800 GB dataset of diverse text," *arXiv:2101.00027*, 2021.

[14] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, "Deduplicating training data makes language models better," in *Proc. ACL*, pp. 8424-8445, 2022.

[15] G. Fowler, L. C. Noll, and P. Vo, "FNV Hash: the FNV non-cryptographic hash algorithm," IETF draft-eastlake-fnv, 2019.

[16] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, vol. 30, pp. 483-485, 1967.

[17] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, pp. 38-45, 2020.

[18] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *NIPS*, pp. 649-657, 2015. AG News: https://huggingface.co/datasets/fancyzhx/ag_news

[19] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007.